SOFTWARE ENGINEERING
MINI-PROJECT

File Compression Tool

Huffman coding in C++17
Part 1 and Part 2 review package

Team E17	SRN
Mahesh Kumar B	PES1UG24CS255
Nishanth T Chigilar	PES1UG24CS302
Mudit Chaturvedi	PES1UG24CS279

Prepared 27 October 2026 | Revision 2.0

Published review candidate: draft PR #13 is linked to the existing sprint board. Linux, macOS, Windows and sanitizer CI jobs passed for code commit fc2db01. Team review and release freeze remain pending. Current demonstrations do not establish historical Sprint-1 recording dates.

Package guide

1. Software requirements specification

2. Software test plan and cases

3. Architecture and design

4. Backlog and two-sprint plan

5. Execution and benchmark evidence

6. Baseline deviations

7. Submission and release checklist

Repository context

Contribution fork: github.com/InterstellarMC/SE-MiniProject-FileCompression. Team upstream: github.com/ChigilarNishanth/SE-MiniProject-FileCompression. The local branch incorporates upstream dc36070. Submitted source documents remain preserved.

Verification at a glance

21 automated cases passed in Release and ASan/UBSan. Two 500 MB fixtures restored with matching SHA-256. ARM64 compression measured about 48 MB/s for text and 33 MB/s for binary. Exact host and acceptance limitations appear in the execution report.

Companion artifacts

Source code, build/CI configuration, individual Markdown documents, backlog.json, raw logs, benchmark_results.json, vector SVG diagrams, and 60/120-second current-product demonstration videos accompany this PDF in the project folder.

PART 1

Software Requirements Specification

Document control

Project: File Compression Tool (czip). Team: E17. Revision: 2.0, prepared 27 October 2026. Status: implementation baseline proposed for team review. The submitted September document is preserved in SRS_Deliverable1.md and SRS_Deliverable1.pdf; this revision does not claim to have been submitted earlier. Structure follows the introduction, overall description, and specific requirements organization of IEEE 830-1998. No certification of standards conformance is claimed.

1. Introduction

1.1 Purpose and audience

Specify observable acceptance criteria for a standalone C++ command-line file compressor, for developers, testers, and the course evaluator. Requirement identifiers retain the submitted SRS identifiers so backlog, design, and tests remain traceable.

1.2 Scope

Compress and decompress one regular file at a time with deterministic, static Huffman coding. The custom .czip container includes metadata, byte frequencies, a bit-packed payload, and CRC-32. Compression is lossless for arbitrary byte values. ZIP interoperability, directory archiving, encryption, network access, and a graphical interface are outside scope. Incompressible or small files may grow because every archive has a 2,072-byte header.

1.3 Definitions and references

FR: functional requirement. NFR: non-functional requirement. CRC: cyclic redundancy check for accidental corruption. SHA-256: cryptographic digest used by tests to compare original and restored files. KiB means 1,024 bytes; MB means 1,000,000 bytes.

References: supplied Software Engineering Mini-Project Deliverables Part-1 and Part-2 briefs; original repository SRS; IEEE 830-1998 (SRS organization), IEEE 829-1998 (test-plan organization), IEEE 1016-2009 (design-description concerns); C++17 language and standard library. These are structural references; normative standards text is not reproduced.

2. Overall description

2.1 Product perspective and operating environment

A local CLI invoked by a user or automated test runner. Requires a C++17 compiler and filesystem supporting sibling temporary directories and hard links. CMake provides a portable build; a direct compiler command is also supported. macOS/Apple Clang is locally validated. Release build/test jobs on Linux, macOS and Windows, plus Linux sanitizers, passed in remote CI for code commit fc2db01; exact run evidence is in remote_validation.json.

2.2 Users and assumptions

Users can run terminal commands and choose input/output paths. Source data must remain stable during compression. Output directories must exist and be writable. The tool rejects non-regular input files. Existing destinations are never overwritten, including the original input path. There is no authentication system because the tool inherits the invoking user's filesystem permissions.

2.3 Constraints and dependencies

No third-party compression library. Source files are processed in two passes during compression and one pass during decompression. Fixed-size buffers and a tree with at most 511 nodes avoid memory proportional to file size. Standard-library filesystem operations require C++17. Default decompression output cap is 1 GiB; a deliberate --max-output BYTES override supports larger trusted archives. Negative savings are valid and reported.

3. Specific requirements

3.1 Functional requirements

ID	Requirement and measurable acceptance criterion
FR-01	czip -c INPUT [OUTPUT] shall produce a version-1 .czip archive and exit 0 for a readable, stable regular file and a writable, absent output. Default output is INPUT.czip.
FR-02	czip -d ARCHIVE [OUTPUT] shall reproduce every input byte from a valid supported archive. Source and restored SHA-256 shall match. Default output removes a final .czip extension, otherwise appends .out.
FR-03	Compression shall count all 256 byte values in unsigned 64-bit counters. Serializing bytes 0..255 repeated four times shall produce 256 counters equal to 4.
FR-04	The min-heap shall order nodes by frequency, then minimum byte value in the subtree; first removed is left child. Identical inputs shall produce identical archives.
FR-05	The header shall contain literal bytes 43 5A (hex), version 1, flags 0, original size, payload bit count, CRC-32, and 256 frequencies, with multibyte integers in little-endian order. Header length shall be 2,072 bytes.
FR-06	Codes shall be packed most-significant bit first. Final unused bits shall be zero. A one-symbol tree shall use code 0. An empty file shall have zero payload bits.
FR-07	Every successful operation shall print original bytes, archive bytes, savings to two decimals, elapsed milliseconds, and output path. Empty-input savings shall be 0.00 percent to avoid division by zero.
FR-08	--help shall print usage to stdout and exit 0. Invalid syntax shall print usage to stderr and exit 2.

3.2 Non-functional requirements

ID	Requirement and measurement
NFR-01	Release compression shall exceed 10 MB/s on the originally specified dual-core x86-64 reference class, measured as input bytes / wall seconds for a 500 MB text fixture and a 500 MB uncompressed binary fixture. The available ARM64 host gives supplementary evidence; the original x86-64 target remains pending.
NFR-02	Peak heap shall stay below 50 MB for inputs up to 500 MB. Fixed buffers and bounded tree provide design evidence; measured peak process RSS below 50 MB is a conservative supporting check, with allocator-specific heap instrumentation still a distinct measurement.
NFR-03	All positive round-trip fixtures shall have identical SHA-256 hashes before compression and after decompression.
NFR-04	Missing files, invalid archives, write failures, and output conflicts shall produce a nonempty stderr diagnostic and no published partial destination.
NFR-05	Exit status shall be 0 on success, 1 on execution/integrity failure, and 2 on invalid command syntax. Security identifiers shall be diagnostic categories, not process exit statuses 101/102.

4. Security

4.1 Objectives

SEC-OBJ-01: detect accidental archive corruption and truncation before publishing restored data. A CRC is not authentication; an adversary who rewrites metadata and checksum can create another valid archive. Cryptographic authenticity is outside scope and the original absolute tamper-detection statement is clarified accordingly.

SEC-OBJ-02: reject malformed input without unsafe memory traversal, uncontrolled allocation, or modification of an existing output. Memory-safety validation combines bounded algorithms, negative tests, and AddressSanitizer/UndefinedBehaviorSanitizer.

4.2 Requirements

SEC-REQ-01: before building the tree, the decoder shall validate signature, version, flags, declared size against the output cap, and frequency sum using overflow-safe arithmetic. Invalid values shall exit 1 with SEC_ERR_BAD_HEADER.

SEC-REQ-02: the decoder shall reject inconsistent bit counts, unexpected archive lengths, invalid one-symbol codes, nonzero padding, frequency mismatches, wrong decoded size, and CRC mismatch with SEC_ERR_BAD_HEADER or SEC_ERR_MALFORMED_STREAM as appropriate. It shall publish no output on failure and clean up provisional data on handled errors.

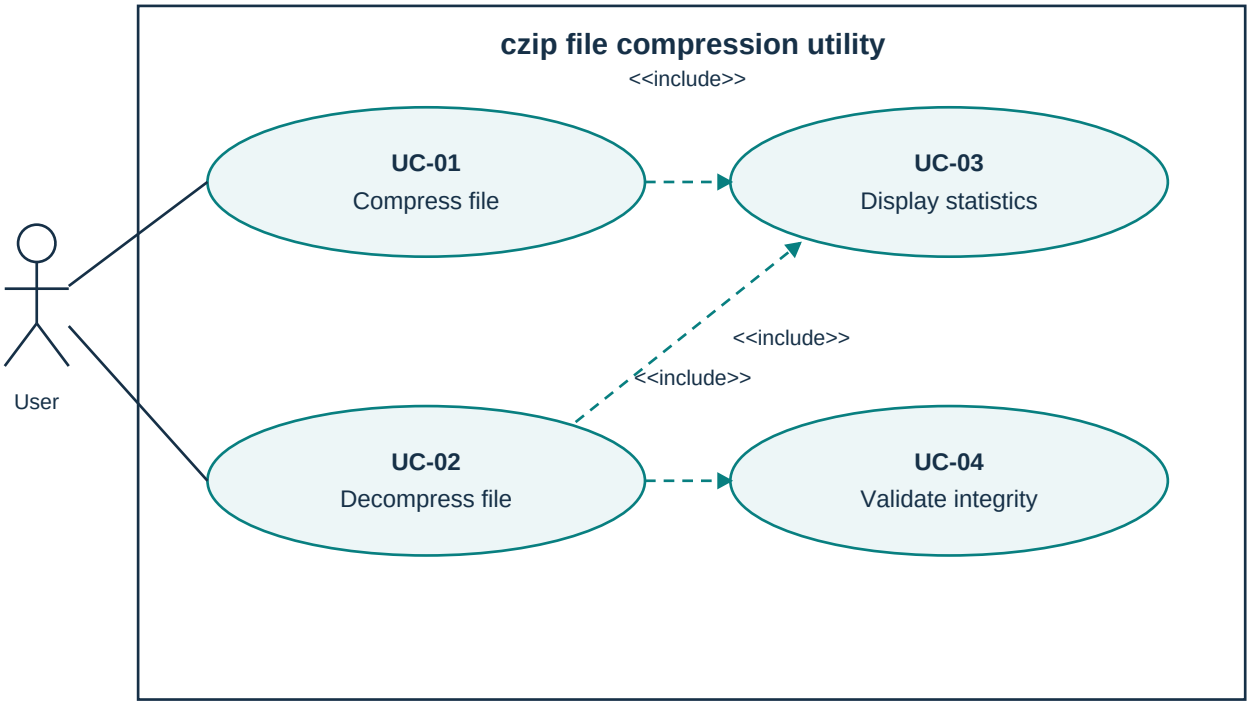
SEC-REQ-03 (added): the encoder and decoder shall refuse an existing output and publish through an atomic no-replace hard link only after successful validation and close. A 1 GiB default output limit shall bound expansion; --max-output permits an explicit numeric limit.

5. Interfaces and use cases

5.1 External interfaces

CLI forms: czip -c INPUT [OUTPUT]; czip -d INPUT [OUTPUT] [--max-output BYTES]; czip --help. The limit is a decimal unsigned integer, including zero, accepted only in decompression mode and at most once. Paths containing spaces are passed as one shell argument. Source paths may begin with a hyphen because INPUT is positional. There are no network or service APIs.

5.2 Use-case diagram



Host filesystem supplies input, staging and final publication.

5.3 Use cases

UC-01 Compress: user supplies a stable file; the system counts frequencies and CRC, constructs a deterministic tree, writes header and packed payload to provisional output, confirms second-pass content consistency, publishes, and prints statistics. On failure, exit 1 with diagnostic and remove provisional output.

UC-02 Decompress: user supplies archive and output limit; the system validates metadata, rebuilds the tree, decodes into provisional output, checks decoded count/frequencies/CRC/padding, publishes, and prints statistics. Invalid archives fail without a destination file.

UC-03 Display statistics is included by both successful operations. UC-04 Validate integrity is included by decompression. The host filesystem supplies reads, provisional writes, and atomic no-replace publication.

6. Acceptance and change control

All automated tests must pass; no sanitizer diagnostics are accepted. Benchmark evidence must state host, build, fixture sizes, and measurement limitations. Release review must include unresolved platform/performance evidence and the deviation log. Team review, participant-authored PRs, reviewer approval, and historical sprint evidence cannot be inferred from locally generated files.

PART 2

Software Test Plan and Test Case Specification

1. Test plan identifier and introduction

TP-CZIP-2.0, Team-E17, prepared 27 October 2026. IEEE 829-style test-plan sections cover test items, tested/excluded features, approach, acceptance, suspension, deliverables, tasks, environment, responsibilities, schedule, risks, and approvals. This plan validates the revised SRS against executable behavior. Original course minimum: 7-10 test cases; this baseline supplies 21 automated cases plus two non-functional benchmark cases.

2. Test items

Release and sanitizer builds of src/czip.cpp; CLI parser/controller, frequency table, min-heap and codebook, version-1 header, packed bitstream, CRC validation, bounded decoder, transactional output, and statistics. Configuration items: CMakeLists.txt, .github/workflows/ci.yml, tests/test_czip.py, and tools/benchmark.py. Use only generated, non-sensitive fixtures in temporary directories.

3. Features to be tested

All FR-01 through FR-08; NFR-01 throughput, NFR-02 memory, NFR-03 SHA-256 losslessness, NFR-04 feedback, NFR-05 exit statuses; SEC-REQ-01 header bounds, SEC-REQ-02 decoder safety, SEC-REQ-03 output preservation and expansion limits. Edge cases include empty input, one symbol, every byte value, high-entropy binary, ties, malformed headers/payload, invalid syntax, output conflicts, and unavailable output directories.

4. Features not to be tested

ZIP compatibility, encryption, directories, GUI, networking, parallel coding, and unsupported container versions. Local validation does not establish Linux/Windows compatibility, the original x86-64 performance target, or exhaustive resistance to every malicious input. CRC tests do not prove cryptographic authenticity. Power-loss recovery and hostile local filesystem races are outside the acceptance scope.

5. Test approach

Use black-box executable tests so the checks exercise actual file formats, output bytes, exit codes, and publication behavior. Positive tests compare SHA-256 digests or exact bytes; header tests inspect explicit integers; negative tests assert failure, stderr category, absent destination, and cleanup. Deterministically seeded round trips and bit mutations extend fixture coverage without unstable randomness. Repeat the suite under ASan/UBSan. Measure performance separately with an optimized non-sanitized executable, 500,000,000-byte text and binary files, wall-clock time, and SHA-256.

5.1 Security validation

Invalid magic/version/flags, truncated headers, impossible frequency sums, oversized declared output, invalid bit counts, truncated/trailing payload, checksum or payload corruption, nonzero padding, invalid single-symbol codes, and 40 seeded one-bit archive mutations must all fail cleanly. The test harness gives each command a 30-second timeout, rejects crashes through expected exit status, and checks for no leaked provisional directory. Existing output and source aliases must retain their bytes. Sanitizer findings fail the run even if ordinary positive output looks correct. These are negative-input tests, not a claim of exhaustive penetration testing.

5.2 Requirement-to-test traceability

Requirement	Automated case(s) / evidence
FR-01	TC-01, TC-02, TC-03, TC-04, TC-05, TC-17, TC-18
FR-02	TC-01 through TC-05, TC-16, TC-17, TC-18
FR-03	TC-02, TC-06
FR-04	TC-06 deterministic output; TC-18 varied symbol distributions; TC-21 independent golden archive
FR-05	TC-06, TC-11, TC-12
FR-06	TC-03, TC-04, TC-06, TC-13, TC-15
FR-07	TC-07
FR-08	TC-08, TC-17
NFR-01	TC-22, benchmark_results.json; x86-64 reference host pending
NFR-02	TC-23, fixed-memory design and process RSS evidence
NFR-03	TC-01 through TC-05, TC-18, TC-22 SHA-256
NFR-04	TC-09, TC-10, TC-11 through TC-16, TC-20
NFR-05	TC-08, TC-09, TC-11 through TC-16
SEC-OBJ-01 / SEC-REQ-01	TC-11, TC-12, TC-16, TC-19
SEC-OBJ-02 / SEC-REQ-02	TC-11 through TC-15, TC-19; sanitizer log
SEC-REQ-03	TC-10, TC-16, TC-20

6. Test case specification

Each case starts in a fresh temporary directory with an available compiled executable. The run_cli helper checks the expected exit status; round_trip writes input, compresses it, restores it, and compares SHA-256. Case numbers correspond to test method prefixes.

Case	Input and procedure	Expected result
TC-01	Repeated ASCII text; compress then decompress	Exit 0; restored SHA-256 equals source
TC-02	All 256 byte values repeated 37 times; round trip	Binary-safe byte identity
TC-03	Empty file; round trip	Valid 2,072-byte header-only archive; empty restored file
TC-04	10,001 bytes of FF hex; round trip	Single-symbol coding decodes exactly
TC-05	Seeded 100,000-byte random input; round trip	Valid archive even if size grows; exact restoration

Case	Input and procedure	Expected result
TC-06	Bytes 0..255 repeated four times; compress twice; inspect header	Identical archives, literal CZ/version/flags, original size 1,024, all 256 counters = 4
TC-07	40,000 A bytes; inspect success statistics	Archive smaller than input; original/archive/savings/time labels present
TC-08	Help, missing/unknown flags, invalid/duplicate output caps	Help exit 0 to stdout; syntax errors exit 2 with usage on stderr
TC-09	Compress a nonexistent file	Exit 1; IO_ERROR diagnostic
TC-10	Use source as destination; restore into an existing file	Exit 1; source and existing destination remain unchanged
TC-11	Alter signature/version/flags; truncate header at 0, 1, 12, 2,071 bytes	Exit 1; SEC_ERR_BAD_HEADER; no destination or staging residue
TC-12	Set size, bits, or first frequency to uint64 maximum	Exit 1; reject header/cap/overflow inconsistency
TC-13	Delete final payload byte or append one byte	Exit 1; SEC_ERR_MALFORMED_STREAM; exact-length enforcement
TC-14	Flip checksum byte and packed payload byte independently	Exit 1; integrity failure; no published data
TC-15	Corrupt zero padding or one-symbol code	Exit 1; malformed stream rejection
TC-16	Restore 100-byte input with cap 99, then cap 100	First rejected without output; second succeeds exactly
TC-17	Paths containing spaces; omit output argument in both modes	Default naming works and restores correct contents
TC-18	25 seeded varied-length/alphabet fixtures	Every round trip returns identical bytes
TC-19	40 seeded single-bit mutations across header/payload	Every mutation rejected without crash or leftover files
TC-20	Output under a nonexistent parent directory	Exit 1; no provisional directory remains
TC-21	Independently construct AAABBC header and known payload 1F 00; compare encoder and decode oracle	Exact archive bytes and restored content match independent specification
TC-22	Release build; 500 MB text and binary; wall-clock timing and hash checks	Compression >10 MB/s on specified reference class; identical restored SHA-256. ARM64 result reported separately
TC-23	Record peak process RSS during TC-22 and inspect bounded allocations	Supporting RSS <50,000,000 bytes; no growth proportional to input; heap-specific measurement distinguished

7. Item pass/fail criteria

All 21 automated cases must pass for both Release and sanitizer builds; nonzero unexpected exits, timeout, hash mismatch, clobbered target, published corrupt data, or sanitizer output are failures. Performance acceptance needs the stated reference environment; a fast different host is supporting evidence. Memory acceptance uses bounded-allocation inspection plus RSS evidence, with heap-specific instrumentation recommended for formal sign-off.

8. Suspension and resumption

Suspend release review after a data-loss bug, reproducible sanitizer finding, failed hash comparison, or overwritten source. Preserve the failing fixture and log in an issue. Resume after correction, targeted regression, and the full relevant requirement suite. Do not replace missing evidence with a passing declaration.

9. Test deliverables

Source tests, Release log test_results_release.txt, sanitizer log test_results.txt, benchmark_results.json, Test_Report.md, and demo command transcripts. Generated fixtures and build binaries stay outside Git. CI logs become separate remote evidence only after execution.

10. Tasks and environment

Build Release, run the requirement suite, build with -fsanitize=address,undefined and run the suite, execute 500 MB benchmark, inspect report, render submission PDF, verify diagrams and page layout, and run current-product demonstrations. Tests require Python 3.9+ for randbytes; CI selects Python 3.12. Optional media/PDF tools are authoring dependencies, not application runtime dependencies.

11. Responsibilities and schedule

Use the existing upstream story ownership: Nishanth handles CLI/header/integration/tests; Mahesh handles stream analysis/CI/decoder; Mudit handles tree/bit-packing/statistics. These are backlog responsibilities, not assertions that each person authored the generated patch. Review owner must inspect the actual diff and participant-authored PRs.

Original Sprint 1: 19-23 October 2026. Original Sprint 2: 26-30 October 2026. Current evidence was produced after Sprint 1; the missing historical Sprint-1 record cannot be recreated as contemporaneous evidence. Team review, actual PR author/reviewer evidence, and final freeze remain release tasks.

12. Risks and contingencies

Hard-link support varies by filesystem; validate the evaluation volume or expect explicit failure. Performance depends on CPU, storage, and compiler; record the host instead of extrapolating. Small and incompressible data can expand; explain header overhead in the demo. The new versioned archive rejects scaffold headers; record this change. Unknown GitHub project permissions can prevent board updates; prepared backlog files do not establish a live board.

13. Approvals

Prepared for team and evaluator review. No student signature, instructor acceptance, historical sprint completion, or final release approval is asserted by this generated plan.

PART 3

Software Architecture and Design Specification

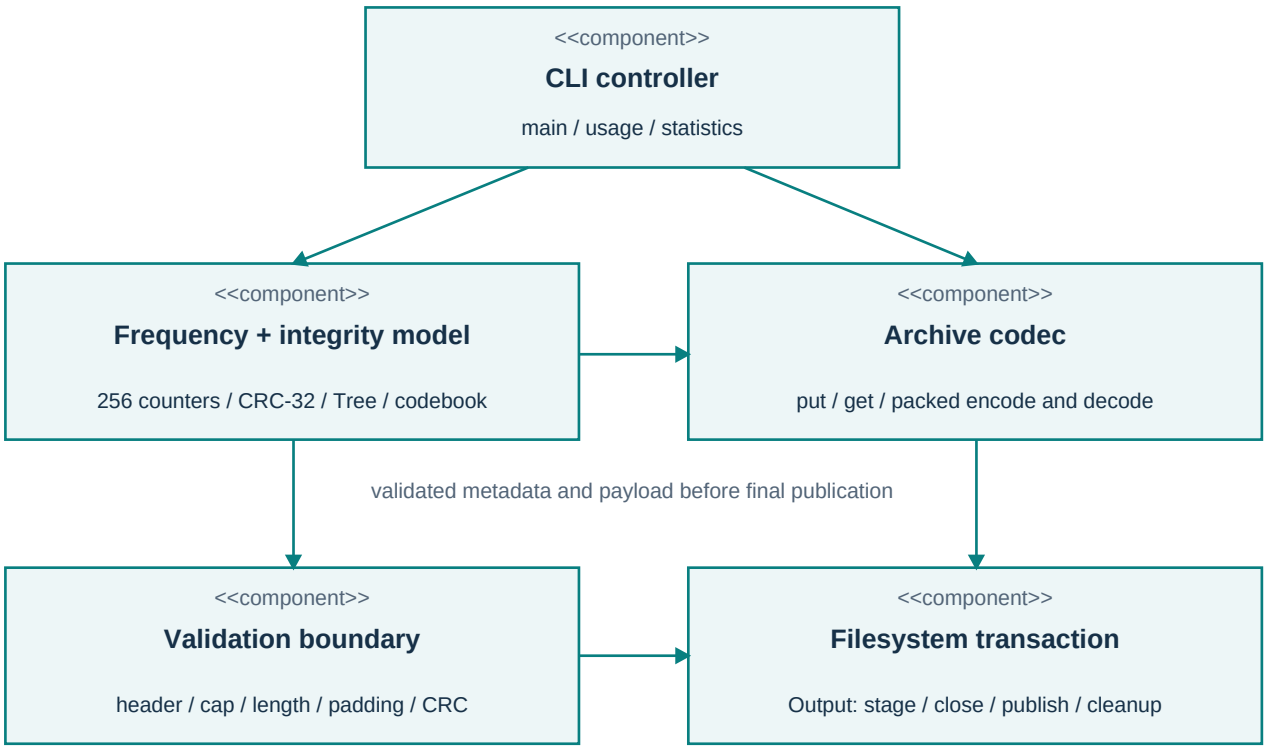
1. Document identification and stakeholders

SAD-CZIP-2.0, Team-E17, 27 October 2026. Status: proposed implementation baseline. IEEE 1016-style organization addresses context, decomposition, interfaces, behavior, and design rationale. Developer concern: deterministic coding and portability. Tester concern: reproducibility and observable failures. Evaluator concern: traceability and working behavior. User concern: lossless restoration and preservation of existing files.

2. Context, pattern, and constraints

The program is a layered processing pipeline: CLI controller -> frequency analysis/tree/codebook -> archive encoder or decoder -> guarded filesystem output. There are no network dependencies or background services. A static Huffman model needs two input passes to encode; decoding rebuilds the same tree from frequencies. C++17 and the standard library satisfy the original C/C++ and no-compression-library constraints.

3. Component view



Logical component	Concrete implementation	Responsibility	Requirements
CLI controller	main, usage	Parse mode, paths, output cap; dispatch; print metrics and exit statuses	FR-07, FR-08, NFR-04, NFR-05
Frequency/integrity analysis	compress, crc_update	Two-pass byte frequency and CRC calculation; source consistency check	FR-01, FR-03, NFR-03
Huffman model	Node, Tree, codebook, bit_count	Priority queue, deterministic tie-break, prefix codes and safe weighted bit count	FR-04, FR-06
Container codec	put, get, compress, decompress	Explicit byte order; header and payload serialization; strict decoding	FR-01, FR-02, FR-05, FR-06
Validation boundary	require and decoder preconditions	Header bounds before tree creation; cap, length, padding, frequency, CRC checks	SEC-REQ-01, SEC-REQ-02
Output transaction	Output	Unique sibling staging directory, flush/close, atomic no-replace publication, cleanup	SEC-REQ-03, NFR-04

These are logical components within src/czip.cpp, not separate translation units. The team scaffold's header-only demonstration is superseded; its original commits and submitted documents remain in Git history. Consolidation keeps the miniature implementation buildable without claiming that the older design's named source modules exist.

4. Data model and file format

Offset (bytes)	Length	Field	Validation
0	2	Literal ASCII C Z (43 5A hex)	Must match exactly
2	1	Version = 1	Unsupported versions rejected
3	1	Flags = 0	Unknown flags rejected
4	8	Original size, uint64 little-endian	At most configured limit
12	8	Number of meaningful payload bits	Must equal weighted code lengths
20	4	CRC-32/ISO-HDLC of original bytes	Reflected polynomial EDB88320, init/final XOR FFFFFFFF
24	2048	256 uint64 little-endian frequencies	Overflow-safe sum equals original size
2072	ceil(bits / 8)	Huffman payload, MSB first	Exact length; zero padding; full decode

No filenames or paths are embedded in the archive, so archives cannot select extraction destinations. Empty inputs use size=0, bits=0, CRC=0, all frequencies zero. A one-symbol input gets one 0 bit per original byte. Small or uniform inputs may yield negative savings. This is a custom versioned container and is not compatible with PKZIP/unzip or the team's earlier native-layout scaffold headers.

5. Algorithm and complexity

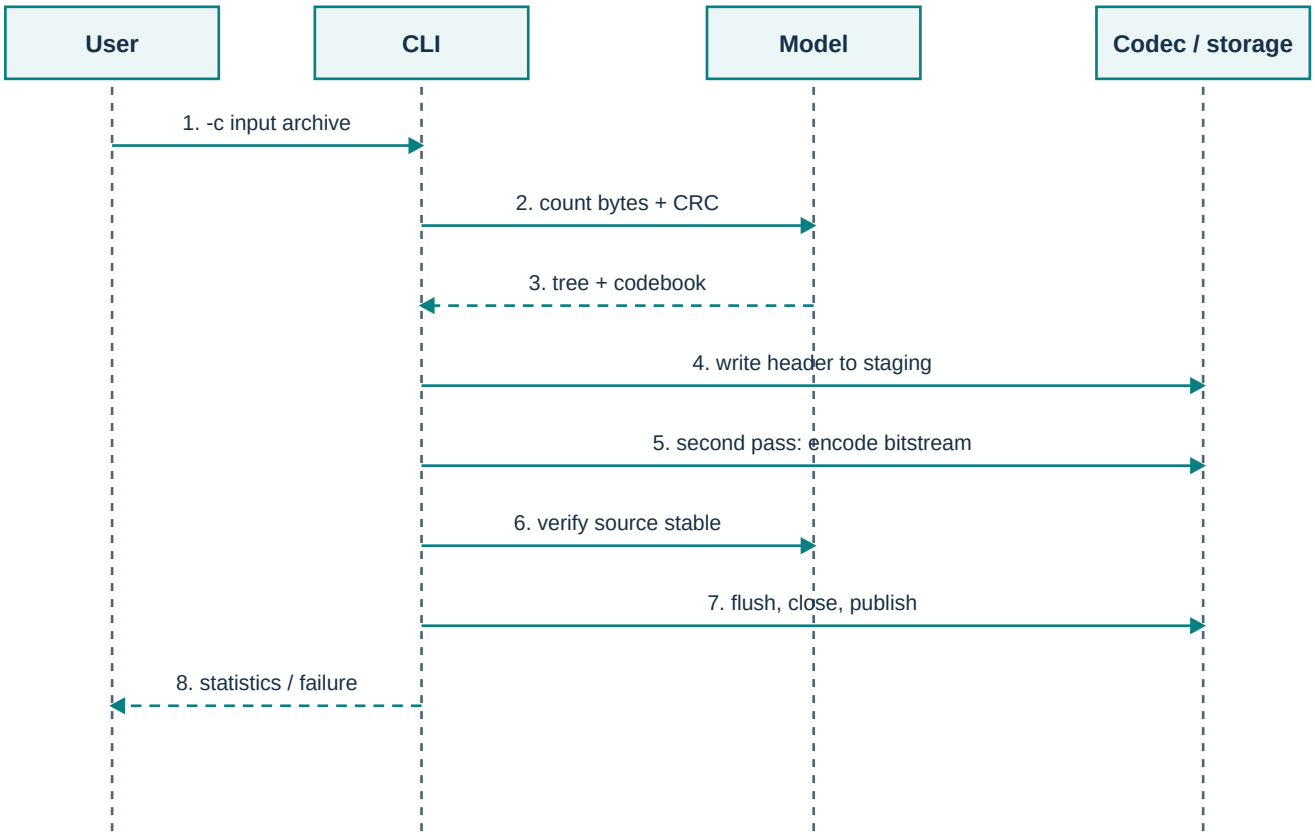
Count byte frequencies with a 64 KiB read buffer. Insert nonzero symbols into a min-heap ordered by (frequency, minimum descendant byte). Remove the two smallest nodes, attach first as left (0) and second as right (1), and reinsert their parent until one root remains. Traverse the tree to form prefix-code vectors, giving the single leaf a 0 code. Weighted payload bits equal sum(frequency[symbol] * code_length[symbol]); guard multiplication and addition from overflow.

Encoding reads the file again and packs codes MSB first; both passes must have matching byte count, frequencies, and CRC before publication. Decoding validates all metadata first, reconstructs the deterministic tree, walks one bit at a time, checks each symbol's expected frequency, and verifies final boundaries and CRC. Nodes are owned by unique_ptr, so exception unwinding frees the entire tree.

For n bytes and alphabet size k <= 256: time O(n * maximum_code_length + k log k), with maximum depth 255; auxiliary memory O(k squared + buffer_size) because explicit code vectors can store k paths of depth k-1. With fixed k=256 this is bounded independently of n. No whole-file buffers are used. Disk space can temporarily hold the full proposed output; the decoder's declared-size cap limits expansion by default.

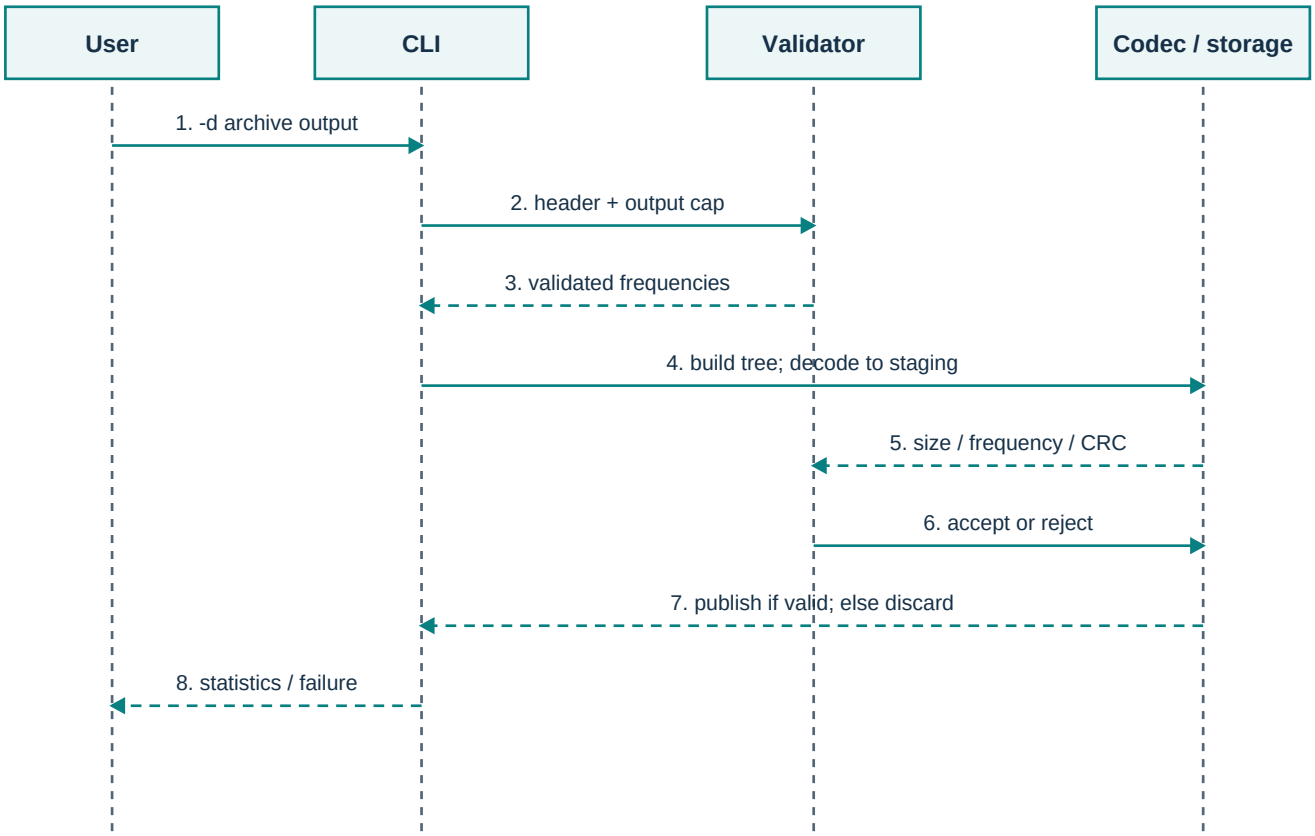
6. Behavioral views

6.1 Compression sequence



Failure: exit 1, no published destination, provisional output cleaned up.

6.2 Decompression sequence



Failure: exit 1, no published destination, provisional output cleaned up.

7. Internal API design

Interface	Inputs	Return / side effect	Failure
compress(source, destination)	Regular file and absent output path	Stats {original, archive}; published archive	Failure exception on I/O, conflict, or changed source
decompress(source, destination, limit)	Archive path, absent output, uint64 byte cap	Stats; validated restored file	Failure on malformed data, cap, I/O, conflict
Tree(frequencies), codebook()	Validated 256 counters	Owned tree and code vectors	Allocation exception propagated
bit_count(frequencies, codes)	Frequencies and code lengths	uint64 weighted bit count	Overflow rejected
put(stream, value, bytes), get(stream, bytes)	Integer width 1, 4, or 8	Serialized/read integer	get rejects premature EOF
Output(destination), commit()	Absent target and existing writable parent	Provisional stream; atomic final publication	Fail without replacing target
require(condition, category)	Boolean invariant and diagnostic	Continues or throws Failure	Converted to stderr and exit 1 by main

No REST API is needed for a local utility. The CLI is its supported public interface; internal function signatures are implementation interfaces.

8. Error handling and security architecture

Validate signature/version/flags/size/frequencies before allocating tree nodes. Validate bit arithmetic and archive length before creating provisional output. Decode to a file owned by an exclusively created sibling directory; check data integrity before committing. create_hard_link fails atomically if any target already exists, covering the race between the initial existence check and publication. The destructor removes provisional files on handled errors. A forced process kill may leave a .czip-tmp-* directory for manual removal.

The threat model covers malicious archive bytes, truncation, malformed metadata, output conflicts, and accidental source changes. CRC is corruption detection, not protection against intentional forgery. The tool does not encrypt data. A highly privileged local adversary able to alter the process or filesystem is outside this model. Source/parent path symlink changes by an adversarial local user are not fully defended with descriptor-relative APIs. Disk quota exhaustion is reported as an I/O failure; no disk reservation is promised.

The output publication mechanism requires a hard-link-capable local filesystem. Unsupported filesystems fail explicitly. The output cap bounds decoded bytes, but the user can deliberately relax it; trusted large archives still need adequate time and disk space.

9. Build, deployment, and verification

Build with CMake Release or c++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror src/czip.cpp -o build/czip. No runtime installation or server deployment is required. CMake's sanitizer option enables ASan/UBSan for GCC/Clang. CI defines Release build/test jobs for Linux, macOS, and Windows plus a Linux sanitizer job. Remote CI is evidence only after it actually executes; local logs are separately preserved.

10. Traceability and design decisions

FR-01/FR-02 use pipeline stages and transactional output. FR-03/FR-04 use frequency table and deterministic tree. FR-05/FR-06 use explicit-endian codec and bounded traversal. FR-07/FR-08 use CLI controller. NFR-01/NFR-02 motivate buffered streaming. NFR-03 and SEC-OBJ-01 use CRC plus SHA-256 test comparison. SEC-OBJ-02 and SEC-REQ-01/02 use validation before allocation and sanitizer-backed negative tests. The complete requirement-to-test matrix is in the Test Plan.

Decisions: frequency-table storage favors transparency over compact headers; deterministic ties ensure reproducible archives; versioning isolates the earlier scaffold format; no-replace publication favors data preservation over overwrite convenience; bounded streaming supports the 500 MB acceptance target. Future work could add a compact canonical-code header, faster lookup decoding, and explicit cryptographic authentication through a separately reviewed format version.

PART 4

Backlog and Two-Sprint Plan

Prepared 27 October 2026. Existing upstream stories US-01 through US-10 and their assignees are preserved. Story points for US-01 through US-10 preserve the existing board Estimate values. US-01 through US-05 retain their observed Sprint-1 allocation; US-06 through US-10 are currently in Backlog and are proposed for Sprint 2. US-11 through US-15 are proposed additions. The existing linked board is SE Mini-Project Sprint Board, project #2 under ChigilarNishanth.

Existing project: https://github.com/users/ChigilarNishanth/projects/2 (private; observed in the authenticated browser). No new Project is needed.

Target repository: https://github.com/ChigilarNishanth/SE-MiniProject-FileCompression

Sprint 1: 19-23 October 2026; Sprint 2: 26-30 October 2026. Sprint 1 is already past; this plan does not backdate work.

Story	Pts	Sprint	Owner	Requirements	Evidence
US-01 CLI Argument Parser & Help Navigation	2	1	ChigilarNishanth	FR-08, NFR-04, NFR-05	TC-08, TC-17
US-02 File Stream I/O & 256-Byte Frequency Analyzer	3	1	Mahesh33-mahi	FR-03, NFR-02	TC-02, TC-06, TC-22, TC-23
US-03 Min-Heap Priority Queue & Huffman Tree Construction	5	1	InterstellarMC	FR-04	TC-06, TC-18, TC-21
US-04 Archive Header Serialization & Magic Byte Validation (0x435A)	3	1	ChigilarNishanth	FR-05, SEC-REQ-01	TC-06, TC-11, TC-12
US-05 Automated C/C++ CI Build & Testing Workflow	2	1	Mahesh33-mahi	NFR-03, NFR-05	CI matrix
US-06 Bitstream Buffer Packing & Bit-Shift File Writer	5	2	InterstellarMC	FR-06	TC-04, TC-15, TC-21
US-07 End-to-End File Compression Engine Integration	5	2	ChigilarNishanth	FR-01	TC-01 through TC-07, TC-18
US-08 Decompression Engine & Safe Bounds Tree Traversal	5	2	Mahesh33-mahi	FR-02, SEC-REQ-02	TC-01 through TC-05, TC-11 through TC-19
US-09 CLI Benchmark Output & Performance Statistics Display	2	2	InterstellarMC	FR-07, NFR-01	TC-07, TC-22
US-10 SHA-256 Lossless Verification & Automated Test Suite	3	2	ChigilarNishanth	NFR-03, SEC-REQ-02	TC-01 through TC-21
US-11 Streaming performance and memory acceptance	3	2	InterstellarMC	NFR-01, NFR-02	TC-22, TC-23; benchmark_results.json

Story	Pts	Sprint	Owner	Requirements	Evidence
US-12 Output preservation and decompression resource limits	3	2	Mahesh33-mahi	SEC-REQ-03, NFR-04	TC-10, TC-16, TC-20
US-13 Complete SRS, test plan, design and change log	3	2	ChigilarNishanth	FR-01 through FR-08, NFR-01 through NFR-05	docs and submission PDF
US-14 Demonstration evidence and contributor review	3	2	InterstellarMC	Part-2 demonstration and review process	demos; PR and review evidence pending
US-15 Final review and development freeze	2	2	ChigilarNishanth	Part-2 final documentation and freeze	docs/Submission_Checklist.md

Sprint 1 goal and capacity

Planned total: 15 story points. Goal: CLI, frequency model, tree, header validation and CI, consistent with the observed board allocation.

Sprint 2 goal and capacity

Planned total: 34 story points. Goal: packed encoding, complete compression and safe decompression, measured non-functional behavior, documentation, current demonstration, and release review.

Definition of ready and done

Ready: linked requirement, owner, points, measurable acceptance, dependencies, and fixture plan. Done: implementation reviewed, relevant tests pass, remote CI verified, documents updated, demonstration evidence linked, and actual assignee/reviewer participation recorded. Local implementation alone does not close upstream issues.

GitHub publication procedure

Reuse existing issues #1 through #10 in the team repository; do not duplicate them in the fork. Reuse the existing linked Project #2 and its Estimate field. Retain current issue states; add requirement/evidence links and proposed Sprint-2 items after review. Keep status based on actual acceptance, not the old sprint dates. The fork has issues disabled, so it is the contribution source, not the team backlog.

Acceptance records

Per-story acceptance criteria, existing issue links, observed states and proposed ownership are included in docs/backlog.json and the full Markdown sprint plan. The plan does not close existing upstream issues.

PART 5

Test Execution and Benchmark Report

Evidence prepared 27 October 2026. Host: macOS-27.0.1-arm64-arm-64bit. Compiler: Apple clang version 21.0.0 (clang-2100.1.1.101). Release: -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror.

Automated validation

21 automated cases passed in Release and ASan/UBSan builds. Cases include 25 seeded round trips, 40 seeded single-bit archive mutations, and an independently constructed golden archive. Detailed logs are test_results_release.txt and test_results.txt. No remote CI result is implied.

500 MB non-functional measurements

Each fixture contains 500,000,000 bytes. Timing includes complete CLI process execution and I/O; MB/s uses decimal megabytes. Compression uses two source passes. macOS /usr/bin/time -l reports peak process resident set size in bytes, not an allocator-specific heap measurement.

Fixture	Compress MB/s	Decompress MB/s	Archive bytes	Peak process RSS (MB)	SHA-256 match
text	47.98	29.97	285,105,010	1.70	True
binary	32.90	27.55	500,002,072	1.79	True

Interpretation and unresolved evidence

Both fixtures exceed 10 MB/s on the available ARM64 host. NFR-01 specifies an x86-64 reference class, so this is supplementary evidence and that exact environment remains unverified. Peak RSS is below 2 MB on this run and supports the bounded-memory design. RSS is broader than heap; no heap-specific profiler output is claimed.

The binary fixture is a deterministic one-megabyte random block repeated 500 times; static per-byte Huffman coding does not exploit long-range block repetition, and the uniform byte distribution makes its payload effectively incompressible. Its 2,072-byte expansion is expected header overhead. Test results are fixture-specific and not a proof for all inputs.

Reproduction

Build Release, run python3 tests/test_czip.py build/czip, then python3 tools/benchmark.py build/czip 500. For sanitizers, use CMake -DCZIP_SANITIZE=ON or the equivalent compiler flags. Allow roughly 1.5 GB of temporary free disk space for the binary source, archive and restored file.

Remote CI and publication

Draft PR #13 is published and linked to the existing sprint board as Sprint 2 - In Progress. GitHub Actions run 40556449207 passed all four jobs (Linux, macOS, Windows, Linux sanitizers) for code commit fc2db01268a62ec9518eee7499326f28304f7a17. See remote_validation.json for exact URLs. The initial MSVC warning about mixed bool/integer bit operations was corrected with explicit unsigned conversions. The actions now use verified Node-24 releases checkout v7.0.1 and setup-python v7.1.0.

A follow-up 500 MB run of the corrected CMake Release build also restored both fixtures exactly: 37.62 MB/s text compression and 29.17 MB/s binary compression, with peak RSS below 2 MB. This CMake build uses its Release optimization defaults and ran while demonstration media were being rendered; it is additional correctness/resource evidence, not a controlled comparison against the original -O2 measurements. Raw follow-up data are in benchmark_followup_results.json.

PART 6

Baseline changes and deviations

Prepared 27 October 2026 against the submitted Part-1 SRS and upstream main dc36070.

ID	Original statement / implementation	Current treatment	Reason and review impact
DEV-01	Project title says Basic ZIP Implementation	Custom .czip Huffman container, explicitly non-PKZIP	The actual submitted scope already specified .czip; title cannot promise ZIP interoperability
DEV-02	Header fields named without a complete byte-order/version contract; scaffold writes native uint16 0x435A	Literal bytes CZ, version/flags, little-endian sizes/frequencies, bit count and CRC-32	Portable deterministic format and stronger corruption checks; scaffold archives intentionally unsupported
DEV-03	Security objective implies any modification is detectable	Detect accidental corruption; CRC is not authentication	A malicious party can recompute a checksum; cryptographic authenticity needs a separate requirement
DEV-04	Security errors named like return codes, while NFR-05 permits only 0/1/2	Diagnostic categories SEC_ERR_BAD_HEADER / SEC_ERR_MALFORMED_STREAM, process exit 1	Resolves inconsistent exit-code language
DEV-05	Design lists several unimplemented .cpp modules	Logical modules consolidated into src/czip.cpp	Small complete implementation; replaced sample-frequency scaffold; component/API mapping updated
DEV-06	Empty-file test permits warning or generic failure	Deterministic success with empty valid archive and zero-size restored file	Clear measurable behavior without data loss
DEV-07	Help test expects stderr	--help to stdout; syntax errors to stderr	Explicit CLI contract; tests updated
DEV-08	Original NFR-01 targets dual-core x86-64	ARM64 evidence measured locally; x86-64 acceptance left pending	Available host is not the specified reference environment
DEV-09	Original NFR-02 names peak heap	Bounded allocations plus process RSS evidence; formal heap profiling distinguished	RSS is broader than heap and should not be mislabeled
DEV-10	No output conflict or expansion policy stated	No overwrite; staged atomic publication; 1 GiB default cap with override	Protect source files and bound archive expansion; new SEC-REQ-03
DEV-11	Sprint-1 demo requested during 19-23 October	Current-product demonstration prepared after Sprint 1	No fabricated historical recording, member-authored PR, or retrospective completion claim

All changes require team acknowledgement during final review/demo. Original submitted files remain preserved and are not silently replaced.

PART 7

Submission and release checklist

Prepared 27 October 2026. This is a review candidate, not a declaration that the team has frozen or submitted a release.

Part 1 deliverables

Item	Evidence / status
Descriptive SRS and measurable FR/NFR	SRS_v2.md; original submission retained
At least one UML use-case diagram	Combined PDF use-case diagram with actors, use cases and include relationships
Two security objectives and requirements	SRS section 4; three security requirements
Test plan sections 3, 4, 5 and security 5.1	Test_Plan.md
Requirements traceability	Test plan matrix, design mapping, backlog.json
Component diagram, pattern, security architecture	Architecture_Design.md and rendered component view
Two UML sequence diagrams	Compression and decompression views in combined PDF
API and error design	Architecture_Design.md sections 7 and 8
At least 7-10 functional/non-functional cases	21 automated cases plus 2 benchmark cases

Part 2 deliverables

Item	Evidence / status
GitHub project linked to mini-project repo	Existing private SE Mini-Project Sprint Board #2 confirmed linked to the team repository
Backlog derived from FR and NFR	Existing upstream US-01..10 mapped; US-11..15 proposed in backlog.json
Points and team assignment	Existing assignees and Estimate values preserved; proposed additional tasks and Sprint-2 allocations need review
Two sprints planned	Backlog_Sprints.md uses original 19-23 and 26-30 October dates without backdating completion
Working product	Complete C++17 compressor/decoder; local tests and benchmark logs
GitHub Actions	Run 40556449207 passed Linux/macOS/Windows Release and Linux sanitizers for code commit fc2db01
Assigned member raises PR; owner reviews	PR #13 is authored by InterstellarMC; other teammates' acceptance and owner review remain pending
One-minute Sprint-1 outcome video	60-second current-product demonstration supplied; historical Sprint-1 recording not available
Two-minute Sprint-2 outcome video	120-second current-product demonstration supplied
Upload videos to repo	Both demonstration videos published on the contribution branch in draft PR #13
Document deviations	Deviations.md with explicit review impacts
Complete docs and freeze	Review package prepared; team acceptance and final frozen commit pending

Final review procedure

1. Review the actual source, diagrams, test logs, and deviations against the submitted baseline.

2. Publish the contribution branch and open the team's required PR. Retain actual author and reviewer identities; do not impersonate teammates or invent old activity.

3. Run the remote CI matrix and inspect its results. Correct failures before calling all platforms verified.

4. Reuse the existing linked GitHub Project #2, preserve its recorded Estimate values, and review proposed Sprint-2 additions using existing issues wherever possible.

5. Record/confirm the evaluator's acceptance of a current demonstration in place of missing historical Sprint-1 evidence. Historical evidence cannot be manufactured.

6. Validate the x86-64 throughput target or record an approved environment deviation; distinguish process RSS from heap profiling.

7. After owner review and all required acceptance, freeze the approved commit and tag the release. No approval or tag is asserted by this package.

Published review candidate

Draft PR: https://github.com/ChigilarNishanth/SE-MiniProject-FileCompression/pull/13. It is linked to the existing private board as Sprint 2 - In Progress. Existing issue assignments, estimates and completion states are preserved; additional tasks remain proposals in the review package. No team owner approval or merge has been performed.